

PSF-Zero: An Analytic Two-Qubit Gate Synthesizer Combined with a Verified, Ordering-Aware Layout Search for Quantum Circuit Transpilation

TN Holdings
love.os.architect@proton.me
<https://github.com/TN-Holdings-LLC/psf-zero>

2026-09-21

Abstract

We present PSF-Zero, a two-part quantum circuit compiler consisting of (1) an analytic two-qubit gate synthesizer based on the Cartan (KAK) decomposition, implemented as a compiled Rust core, and (2) a qubit layout search built around a specific ordering strategy shown effective in a companion paper. On a fixed 15-qubit, dense-pair-block circuit family, the gate synthesizer reproduces Qiskit's own output to a fidelity of 1.000000000000 at small scale and achieves a 7.0–9.3x cumulative speedup over Qiskit's default `optimization_level=3` synthesis path across independently repeated 10,000-iteration measurements; we report the variance of this speedup honestly, including a 34–54x reduction in timing variance observed between two measurement sessions whose cause we investigated with the actual source diff from both sessions and could not identify. The layout search, verified through the public `compile_for_hardware()` entry point rather than as an isolated component, finds a valid embedding on 26 of 26 saturated test instances—up from 0 of 26 before a feasibility-check defect we found and fixed—with all 26 outputs independently verified structurally valid (zero coupling-map violations) and, at small scale, unitary-equivalent to the input circuit to machine precision (minimum fidelity 0.999999999999994 across 36 independent checks). We further verify the same layout search on two topologies distinct from the square grids used in development, including one (`brick`) on which the search's own earlier ordering strategies had previously found nothing at all. We report every defect found in the course of this verification, including one whose root cause we mischaracterized in an earlier stage of this investigation and later corrected, and one inconsistency in our own reported timing figures that we found and correct in this paper. All code, raw data, and the complete chronological verification record are public.

Introduction

Quantum circuit compilation for near-term hardware combines two largely independent problems: synthesizing an arbitrary computation into a device's native gate set, and mapping the circuit's logical qubits onto physical ones subject to connectivity constraints. PSF-Zero addresses both. Its gate synthesis path replaces Qiskit's own general-purpose two-qubit decomposition with a specialized, analytic

KAK/Cartan decomposition compiled to native code. Its layout path is a prototype search built around a specific empirical finding—that a particular, extremely simple node-ordering strategy resolves a class of instances on which Qiskit’s own production layout pass fails outright—reported in detail in a companion paper [1].

This paper’s contribution is not the ordering insight itself, which belongs to the companion paper; it is the engineering and verification of a system that uses it. We treat "the search finds a layout" as necessary but insufficient, and verify three further properties that, to our knowledge, are not jointly established for any comparable research prototype: that every found layout respects the device’s connectivity (structural validity), that the resulting circuit computes the same operator as the input (unitary equivalence, checked exactly at small scale rather than assumed), and that the underlying search strategy is not overfit to the square-grid topology it was developed on.

We are equally direct about a result we cannot explain. Over the course of this work we observed the gate synthesizer’s own timing variance drop by more than an order of magnitude between two measurement sessions using nominally the same benchmark. We obtained the actual source diff between the two sessions’ codebases, traced every candidate mechanism we could identify, ruled out each one by direct test, and did not find the cause. We report this as a closed, unresolved question rather than an unsupported explanation, consistent with the verification standard we apply to every positive claim in this paper.

Contributions

1. An analytic two-qubit gate synthesizer achieving 7.0–9.3x cumulative speedup over Qiskit’s default synthesis path, with exact correctness verified at small scale (Section 2).
2. An engineered layout search, built on the ordering finding of a companion paper, verified end-to-end through the system’s own public API: 26/26 instance coverage (up from 0/26 before a defect fix), zero structural violations, and unitary equivalence to machine precision (Section 3).
3. Generalization evidence on two topologies outside the search’s own development set, including a topology on which its predecessor strategies previously found nothing (Section 4).
4. A full account of defects found during development and verification, including one root-cause mischaracterization we corrected mid-investigation and one figure-transcription error we correct in this paper (Section 5).
5. An honestly unresolved empirical finding—the gate synthesizer’s own variance reduction across sessions—reported with the investigation that failed to explain it (Section 6).

Gate synthesis

Design

PSF-Zero's `compile()` entry point collects two-qubit blocks from a circuit (via Qiskit's own `Collect2qBlocks` and `ConsolidateBlocks` passes, reused rather than reimplemented) and replaces each consolidated block's unitary with an explicit KAK/Cartan decomposition: two single-qubit unitaries, an entangling core expressed in the target basis (cx in all measurements reported here), and two further single-qubit unitaries. The decomposition itself runs in a compiled Rust extension (`psf_zero_core`), called once per consolidated block.

Correctness

At small scale ($n=6$), where exact operator comparison is tractable, the synthesized circuit reproduces the input circuit's own unitary to a fidelity of 1.000000000000 across all three arms compared (Qiskit `optimization_level=3`, PSF-Zero with and without its own verification pass enabled). We do not compute exact unitary equivalence at the 15-qubit scale used for the timing measurements below; this is the same limitation stated plainly in our own project documentation, and we repeat it here rather than let the small-scale result imply more than it establishes.

Performance

Cumulative time over 10000 iterations, 15-qubit dense-pair-block circuits, `verify=False`, three independent sessions. Standard deviation is per-call, over the same 10000 calls. The archive session's speedup was previously reported as 8.87x in our own project record; recomputed directly from the session's own raw per-call data for this paper, it is 7.00x. We correct the earlier figure here (Section 5) rather than silently using the corrected value without comment.

Session	Total (Qiskit)	Total (PSF- Zero)	Speedup	Std. dev. (PSF- Zero)
Archive (4 days prior)	92.07 s	13.14 s	7.00x	0.902 ms
Today	103.75 s	11.86 s	8.75x	0.123 ms
Repeat	106.56 s	11.49 s	9.27x	0.164 ms

We report a plain speedup range, 7.0–9.3x, rather than a single headline number, because the three independent sessions summarized in Table 1 do not agree to within measurement noise on this specific circuit family and hardware configuration. We consider the range itself, and the reasons behind it (Section 6), more informative than any single figure drawn from it.

Layout search

Background and scope

A companion paper [1] establishes that Qiskit’s VF2Layout pass fails—reporting no solution where a valid embedding exists—on a specific, reproducible class of saturated layout instances, and that a single `rustworkx.vf2_mapping` call with a fixed node ordering (`id_order=True`) resolves every tested instance of that class in microseconds. This paper does not re-derive that finding; it reports the engineering built on top of it and its independent, end-to-end verification.

PSF-Zero’s own layout prototype, `smart_vf2_layout`, predates that finding and originally used a different strategy: a sequence of BFS-family node orderings, tried with `id_order=True`, falling back to `rustworkx`’s own heuristic ordering if all failed. We found and fixed four defects in this prototype in the course of incorporating the companion paper’s finding, summarized in Table 2 and detailed in Section 5.

Defects found and fixed in `smart_vf2_layout`.

Defect	Fix	Effect
Natural ordering (matching the companion paper’s finding) never attempted	Added as the first candidate ordering	56.8x median speedup on the search step
Feasibility guard compared interaction-graph edge count to device matching size, valid only for the prototype’s original circuit family	Compares the interaction graph’s own maximum matching instead	Every chain-shaped circuit family used in this project reaches the search at all for the first time
Guard recomputed the device’s own matching on every call	Cached per device, keyed on graph structure	52.4% reduction in guard cost
Ordering candidates computed eagerly regardless of use	Generator-based, lazy evaluation	Modest additional saving; the larger residual cost was traced to graph construction, not ordering

Is the feasibility guard worth its cost?

The repaired guard adds a fixed per-call cost. We measured directly whether this is worthwhile, using instances specifically constructed to be infeasible by matching capacity but not by raw qubit count—the only case in which the guard can add value, since qubit-count infeasibility is caught by the search itself immediately regardless.

Guard cost versus unguarded search cost on constructed infeasible instances. Two of four searches did not conclude within the 2-second budget used and are reported as lower bounds. The control row confirms the guard is not simply always cheaper—on an instance the search itself rejects immediately, the guard’s own fixed cost exceeds it.

Instance	Guard rejection	Unguarded search
Matching-imbalanced, small	0.70 ms	1642.5 ms
Matching-imbalanced, larger	1.22 ms	≥ 2000.5 ms (budget-limited)
Matching-imbalanced, off-by-one	1.18 ms	≥ 2000.1 ms (budget-limited)
Control (also infeasible by qubit count)	1.00 ms	0.30 ms

Excluding the control case (where the guard is not the relevant mechanism), the guard breaks even against the unguarded search at roughly one infeasible call in 482, using the smallest, fully-concluded search time in Table 3 as a conservative bound.

End-to-end verification

We verified the repaired search through `compile_for_hardware(layout_search=True)`, the system’s own public entry point, rather than through `smart_vf2_layout` alone, on saturated instances at two grid sizes (6x7, 8x8) in two circuit families: the prototype’s original disjoint-pair family, and a chain-shaped family the pre-fix guard rejected outright.

Coverage. All 26 tested instances (matching the companion paper’s own primary instance family) find a valid layout through this entry point, versus 0 of 26 before the guard fix—since the guard previously rejected every one before any search ran.

Structural validity. Every routed output was checked directly against the coupling map’s own edge set. Across all measured configurations, coupling-map violations: 0.

Unitary equivalence. At $n=6$ (both circuit families, three arms, three seeds, two repeats: 36 runs), we compared each output’s operator against the input’s, correcting for the routing-induced qubit permutation using Qiskit’s own `TranspileLayout` interface. The permutation-handling logic itself was independently verified against hand-constructed test cases before use, since an error there could produce a false pass or false fail indistinguishable from a genuine result. Minimum fidelity across all 36 runs: 0.9999999999999994—at the floating-point noise floor, not near any threshold.

Practical benefit. On the chain-shaped family specifically (the instances the guard fix newly unlocks), the repaired search produces a modest but consistent end-to-end

improvement over the default layout path: 1.07x faster at both grid sizes tested, never worse. We report this without embellishment: it is a real but small effect, not the multiple-orders-of-magnitude figure that applies to the isolated search step in Table 2, because the search is a small fraction of the full compilation pipeline’s own cost.

Generalization

Every result in Section 3 used square grids. We tested the repaired search on two structurally different topologies: brick (a lattice on which the prototype’s own earlier BFS orderings had previously found nothing on saturated instances) and heavy-hex (bipartite-imbalanced, so qubit-count saturation is unattainable; we use saturation relative to the lattice’s own maximum matching instead).

Repaired search versus the prototype’s own prior ordering strategies, on topologies neither was developed against.

Topology	n	Repaired search	Prior orderings
brick, 6x7	42	found, 1.25 ms	none found, 875 ms
brick, 8x8	64	found, 1.08 ms	none found, 996 ms
heavy-hex, d=3	19	found, 0.25 ms	—
heavy-hex, d=5	57	found, 0.69 ms	—

The brick result is, to us, the most striking evidence that this is not a square-grid-specific fix: the repaired search succeeds in about a millisecond on instances where the prototype’s entire prior strategy—every BFS-family ordering it previously used, exhausted—found nothing at all.

Defects found

We report every defect found during this investigation, including two that reflect on our own prior conclusions rather than only on the system under test.

The four layout-search defects are summarized in Table 2 above.

A root-cause mischaracterization, corrected. An early stage of this investigation attributed part of the guard’s inefficiency to a specific code pattern (`find_bit`-based qubit resolution) found in an intermediate development revision. Re-reading the actual production codebase directly showed that revision was never deployed; the pattern in question does not appear in the code this paper evaluates. We retract the earlier attribution rather than let it stand alongside the corrected account.

A cumulative-speedup figure, corrected. Table 1 reports the archive session’s own speedup as 7.00x; our own earlier project record reported 8.87x for the same session. We recomputed directly from that session’s own raw per-call data while preparing this paper and found the discrepancy; we do not know whether the earlier figure was a transcription error or drawn from a different run than the archived raw data represents, and we do not guess between these. We use the recomputed, independently-verifiable figure throughout this paper.

A memory-growth question, resolved. Disabling Python’s garbage collector during repeated `compile()` calls revealed measurable memory growth. We traced this to two independent, distinct phenomena: reference cycles in Qiskit’s own circuit objects (not PSF-Zero’s own code), reclaimed cleanly and at a bounded, steady rate under normal (garbage-collection-enabled) operation; and a separate, one-time interpreter warm-up cost in generic runtime objects, confirmed by extending the measurement across multiple checkpoints within a single process and observing growth reach exactly zero after the first several hundred calls. Neither is a genuine leak.

An unresolved finding

Table 1’s standard-deviation column shows a 34–54x reduction between the archive session and the two later sessions (confirmed by Levene’s test, $p \approx 0$ for both comparisons against the archive; the two later sessions do not differ significantly from each other for the `verify=True` arm, $p=0.082$). We investigated this directly rather than let it pass unremarked.

We obtained the actual source code from both sessions and examined the diff line by line. Two changes present between the sessions were traced as candidate explanations and ruled out: a change to the gate synthesizer’s own internal caching eviction policy does not plausibly matter for this benchmark, because the benchmark generates a fresh, uniquely-seeded random circuit every iteration, so the cache provides negligible hit-rate benefit under either eviction policy; and a change eliminating redundant verification work applies only to a different code path (`verify="strict"`) not exercised by this benchmark. The compiled Rust core’s own decomposition routine is unchanged between sessions in every way that could plausibly affect per-call timing. We found no other candidate in the diff.

We did not investigate two remaining candidate explanations: differences in the versions of dependency packages installed on the measurement machine between sessions (not tracked at the time), and machine or operating-system-level environmental factors. We report this as a closed but unresolved investigation rather than speculate further.

Related work

Cartan/KAK decomposition of two-qubit unitaries is well-established [2,3]; our contribution is not the decomposition itself but a compiled, production-oriented implementation and its integration into a verified compilation pipeline. The layout search builds directly on the ordering-sensitivity finding of our companion paper [1]; SABRE [4] remains the relevant point of comparison for heuristic routing when no perfect layout exists.

Conclusion

PSF-Zero combines an analytic gate synthesizer with a layout search engineered around a specific, externally-motivated ordering insight, and we have verified both components more thoroughly than is typical for a research prototype: exact correctness at small scale, structural validity and unitary equivalence end-to-end, and generalization beyond the topology used in development. We have also been direct about what remains unresolved—an unexplained variance reduction in our own timing measurements—and about two mistakes in our own prior reporting that we found and corrected while preparing this paper. We consider this standard of self-correction a necessary part of the contribution, not a caveat to it.

Data and code availability

All raw measurement data, defect reports, and the complete chronological verification record, including every correction discussed in Section 5, are available at <https://github.com/TN-Holdings-LLC/psf-zero>.

9

TN Holdings, *Ordering Sensitivity in Subgraph-Isomorphism Layout Search: A Characterization of Catastrophic Failure Regions in Quantum Circuit Transpilation*, 2026.

N. Khaneja and S. J. Glaser, *Cartan decomposition of $SU(2^n)$ and control of spin systems*, Chemical Physics, 267(1–3):11–23, 2001.

R. R. Tucci, *An introduction to Cartan’s KAK decomposition for QC programmers*, arXiv:quant-ph/0507171, 2005.

G. Li, Y. Ding, and Y. Xie, *Tackling the qubit mapping problem for NISQ-era quantum devices*, ASPLOS ’19, pp. 1001–1014, 2019.